Estructura de la propuesta técnica

Andrés Felipe Montano, Cristian David Yalandad Pillimue, Emmanuel Bonilla Salazar, Erik Santiago Alegría Rojas, Karen Niheidy Pastás Valencia y Karen Vanessa Castañeda Morán

Análisis y Desarrollo de Software, SENA – Servicio Nacional de Aprendizaje

Negociación: C3312932

César Marino Cuéllar Chacón

25 de Marzo del 2026

Tabla de contenido

Introducción 6

1.0 Hoja de control 7

1.1 Definición técnica 7

1.2 Propósito dentro del proyecto 7

1.3 Implementación en el proyecto SENALingua 8

2.0 Equipo desarrollador 9

2.1 Definición técnica 9

2.2 Importancia estratégica 9

2.3 Estructura del equipo SENALingua 10

2.4 Explicación detallada de cada rol 11

3.0 Registro de cambios 14

3.1 Definición técnica 14

4.0 Descripción del proyecto 16

4.1 Problema identificado 17

4.2 Solución propuesta 18

5.0 Justificación del proyecto 19

5.1 Definición técnica 19

5.2 Justificación estructurada 19

6.0 Alcance 22

6.1 Definición técnica 22

6.2 Alcance funcional 22
6.3 Alcance técnico 23
6.4 Alcance organizacional 24
6.5 Exclusiones del alcance 25
7.0 Requerimientos del sistema 25
7.1 Definición técnica 25
7.2 Requerimientos funcionales 26
7.3 Requerimientos no funcionales 30
8.0 Arquitectura tecnológica 32
8.1 Frontend 32
8.2 Backend 33
8.3 Base de datos 34
8.4 Módulo de inteligencia artificial (Chatbot) 35
8.5 Control de versiones 36
8.6 Despliegue del sistema 36
8.7 Estructura general de la arquitectura 37
9.0 Metodología de desarrollo 38
9.1 Definición técnica 38
9.2 Metodología seleccionada 38
9.3 Estructuración operativa de SCRUM en SENALingua 38

- Gestión del producto backlog 38

9.4 Gestión de calidad dentro de Scrum 42

9.5 Gestión de riesgos 42

9.6 Justificación integral de la metodología 43

10. Cronograma del proyecto 43

10.1 Definición del horizonte temporal 43

10.2 Estructura general del cronograma 44

10.3 Distribución por sprints 45

10.3.1 Explicación de la distribución de los sprints 47

11. Propuesta financiera 49

11.1 Mano de obra del proyecto (Costo directo principal) 49

11.2 Costos del hardware (CAPEX) 50

11.3 Costos de software y servicios (OPEX) 51

11.4 Gastos operativos del proyecto 52

11.5 Costos indirectos obligatorios 53

11.6 Utilidad académica del proyecto 54

12. Forma de pago 55

13. Entregables 56

13.1 Documento de requisitos 57

13.2 Código fuente 58

13.3 Manual técnico 59

13.4 Manual de usuario 60

13.5 Informe de pruebas 61

Conclusión 63

Introducción

El presente documento contiene la estructura de la propuesta técnica para el desarrollo del sistema SENALingua, una solución digital orientada al fortalecimiento de competencias en el idioma inglés mediante el uso de tecnologías de información. El proyecto busca proporcionar una herramienta funcional que facilite el acceso a contenidos, ejercicios y recursos que contribuyan al proceso de aprendizaje, promoviendo el uso de entornos digitales como apoyo a la formación académica.

La propuesta integra de manera organizada los componentes necesarios para el desarrollo del sistema, incluyendo la definición de requerimientos, la arquitectura tecnológica, la planeación de actividades, la estimación de recursos, la estructuración financiera, la forma de pago y la definición de entregables. Cada elemento se encuentra alineado con el alcance funcional del proyecto, garantizando coherencia entre las necesidades identificadas y la solución planteada.

La formulación técnica permite establecer una relación clara entre los objetivos del sistema y los medios necesarios para su desarrollo, asegurando que la selección de tecnologías, la organización del trabajo y la distribución de recursos permitan construir un producto funcional, verificable y documentado. La estructura presentada facilita la comprensión del proceso de desarrollo y proporciona una base organizada para la evaluación del proyecto.

La solución propuesta se fundamenta en el uso de herramientas accesibles, metodologías estructuradas y una planificación adecuada de los recursos disponibles, permitiendo asegurar consistencia en el desarrollo del sistema y claridad en la definición de sus componentes.

1.0 Hoja de control

1.1 Definición técnica

La hoja de control es un componente de gobernanza documental que permite identificar, estructurar, versionar y validar formalmente la propuesta técnica dentro del ciclo de vida del proyecto. Su función es garantizar que toda la información contenida en el documento sea trazable, verificable y aprobada por los responsables correspondientes.

1.2 Propósito dentro del proyecto

Elemento	Propósito	Justificación	Impacto
Identificación del documento	Permite reconocer formalmente el entregable	Evita confusión con otros documentos del proyecto	Orden documental
Control de versiones	Registra cambios en el tiempo	Evita uso de información desactualizada	Reduce errores operativos
Autores	Define responsables del contenido	Permite trazabilidad y responsabilidad directa	Control de calidad
Aprobadores	Valida el documento	Garantiza alineación institucional	Gobernanza

Fecha	Contextualiza temporalmente	Permite seguimiento del proyecto	Auditoría

1.3 Implementación en el proyecto SENALingua

Campo	Valor	Explicación
Empresa / Organización	SENA – Centro de Teleinformática y Producción Industrial	Define el marco institucional que respalda el proyecto
Proyecto	SENALingua	Identificador único del sistema
Entregable	Propuesta técnica del software	Documento base para aprobación del proyecto
Autores	Equipo de desarrollo	Responsables directos
Versión / Edición	1.0	Primera versión formal del documento
Fecha	23 de Marzo del 2026	Momento de elaboración
Aprobado por	Pendiente	Será validado por instructor/líder

Número total de páginas	Pendiente	Permite dimensionar el alcance documental y facilitar control de integridad del entregable

Justificación financiera

- La implementación de control documental reduce costos asociados a:
- Retrabajo (repetición de actividades)
- Errores por uso de versiones incorrectas
- Falta de alineación entre stakeholders

Conclusión

El control documental es una práctica de bajo costo con alto impacto en eficiencia, lo que lo convierte en un elemento financieramente obligatorio.

2.0 Equipo desarrollador

2.1 Definición técnica

El equipo desarrollador representa la estructura organizacional del proyecto, donde cada integrante cumple un rol específico orientado a garantizar la planificación, ejecución, control y aseguramiento de calidad del software.

2.2 Importancia estratégica

El recurso humano es el principal activo del proyecto, representando aproximadamente: **60% – 80% del costo total del proyecto**

Por esta razón, una correcta asignación de roles permite:

- Optimización de costos
- Reducción de riesgos
- Mejora en la calidad del producto

2.3 Estructura del equipo SENALingua

Rol	Nombre	Responsabilidad principal
Product Owner	Cristian David Yalandad Pillimue	Definir valor del producto
Scrum Master	Karen Niheidy Pastás Valencia	Gestionar metodología ágil
Diseñador UX/UI	Erik Santiago Alegría Rojas	Diseño de experiencia de usuario
Desarrollador Frontend	Karen Vanessa Castañeda Morán	Interfaz de usuario
Desarrollador Backend	Emmanuel Bonilla Salazar	Lógica del sistema
QA / Tester	Andrés Felipe Montano	Aseguramiento de calidad

2.4 Explicación detallada de cada rol

Product Owner - Cristian David Yalandad Pillimue	
Aspecto	**Descripción**
Función	Definir qué se debe construir
Propósito	Alinear el sistema con los requerimientos del usuario
Justificación	Evita desarrollar funcionalidades innecesarias
Impacto financiero	Reduce desperdicio de recursos

Justificación

El Product Owner es quien traduce el levantamiento de requerimientos en valor real para el producto, asegurando que cada funcionalidad tenga un propósito claro.

Scrum Master – Karen Niheidy Pastás Valencia	
Aspecto	**Descripción**
Función	Gestionar el proceso ágil
Propósito	Garantizar cumplimiento del cronograma
Justificación	Reduce bloqueos y retrasos

Impacto financiero	Evita sobrecostos por retrasos

Justificación

Este rol no desarrolla software, pero es clave para la eficiencia del equipo. Su ausencia suele generar desorganización y retrasos.

Diseñador UI/UX – Erik Santiago Alegría Rojas	
Aspecto	**Descripción**
Función	Diseñar la experiencia e interfaz del usuario
Propósito	Garantizar usabilidad, accesibilidad y satisfacción del usuario
Justificación	Una mala experiencia de usuario reduce la adopción del sistema
Impacto financiero	Incrementa la retención y maximiza el retorno de la inversión

Justificación

El Diseñador UX/UI es responsable de estructurar la interacción entre el usuario y el sistema, asegurando que el producto sea intuitivo, eficiente y alineado con las necesidades reales del usuario. Su

intervención reduce costos asociados a rediseños, soporte técnico y abandono del sistema, impactando directamente en la viabilidad financiera del proyecto.

Desarrolladora Frontend - Karen Vanessa Castañeda Morán	
Aspecto	**Descripción**
Función	Desarrollo de interfaz
Propósito	Permitir interacción del usuario
Justificación	La usabilidad define la adopción del sistema
Impacto financiero	Reduce abandono del producto

Justificación

Un sistema sin buena interfaz no se usa, lo que representa pérdida total de la inversión.

Desarrollador Backend - Emmanuel Bonilla Salazar	
Aspecto	**Descripción**
Función	Desarrollo de lógica y APIs
Propósito	Procesar datos y reglas del sistema
Justificación	Es el núcleo funcional del software
Impacto financiero	Fallas aquí generan costos críticos

Justificación

El backend garantiza que el sistema funcione correctamente, maneje usuarios y procese información.

QA / Tester – Andrés Felipe Montano	
Aspecto	**Descripción**
Función	Validar calidad
Propósito	Detectar errores antes de producción
Justificación	Corregir errores temprano es más barato
Impacto financiero	Reduce costos hasta 10x

Justificación

Un error detectado en producción puede afectar usuarios, reputación y costos de mantenimiento.

3.0 Registro de cambios

3.1 Definición técnica

Es un mecanismo de control que documenta todas las modificaciones realizadas al documento o proyecto, garantizando trazabilidad y control.

Propósito	
Función	**Explicación**
Control	Saber qué cambió
Trazabilidad	Identificar responsables
Auditoría	Validar evolución

Justificación

El registro de cambios no solo permite identificar modificaciones, sino que, además, garantiza coherencia en la evolución del proyecto. En efecto, en entornos reales de desarrollo, los cambios son constantes; por tanto, su control evita desviaciones del alcance.

Asimismo, este mecanismo facilita la trazabilidad de decisiones, lo que permite entender por qué se realizaron ciertos ajustes. En consecuencia, se mejora la gestión del proyecto, ya que cada cambio queda justificado, validado y alineado con los objetivos definidos.

Implementación			
Versión	**Cambio**	**Responsable**	**Fecha**
1.0	Creación inicial	Equipo	23 de Marzo del 2026
2.0	Ajuste financiero	Analista	Pendiente
3.0	Validación técnica	Instructor	Pendiente

Justificación

La implementación mediante versiones estructuradas responde a prácticas estándar en la industria del software. En primer lugar, permite organizar la evolución del proyecto de forma clara; además, facilita identificar qué cambios corresponden a cada fase.

Por otro lado, la asignación de responsables introduce control directo sobre cada modificación, lo cual reduce errores y ambigüedades. En consecuencia, se mejora la calidad del documento y se optimiza la gestión del tiempo dentro del equipo.

Conclusión

El registro de cambios reduce costos asociados a retrabajo y desorganización. En efecto, los cambios no controlados generan reprocesos que incrementan el uso de recursos.

Además, al detectar desviaciones de forma temprana, se evita invertir tiempo en actividades innecesarias. Por tanto, se mejora la eficiencia operativa.

En consecuencia, este mecanismo, aunque simple, contribuye directamente a la optimización del presupuesto y a la sostenibilidad del proyecto.

4.0 Descripción del proyecto

La definición del proyecto SENALingua permite establecer con claridad su propósito dentro del contexto del SENA, específicamente en la formación de aprendices en áreas tecnológicas. En efecto, al tratarse de una plataforma orientada al aprendizaje de inglés técnico, es necesario delimitar su alcance desde el inicio.

Además, esta definición facilita que instructores, desarrolladores y stakeholders comprendan que el sistema no es un curso de inglés general, sino una herramienta especializada en terminología y

contextos propios del software. Por tanto, se asegura coherencia en el desarrollo y en la toma de decisiones.

4.1 Problema identificado

Problema identificado		
Problema	**Explicación**	**Impacto**
Bajo nivel de inglés técnico	Aprendices no dominan terminología	Baja empleabilidad
Falta de herramientas	No hay plataformas especializadas	Aprendizaje limitado
Métodos tradicionales	No son adaptativos	Baja retención

En el contexto del SENA, particularmente en programas de formación en software, se evidencia que muchos aprendices presentan dificultades para comprender documentación técnica, librerías y entornos de desarrollo debido al bajo dominio del inglés especializado.

Asimismo, las herramientas actuales de aprendizaje no están enfocadas en el vocabulario técnico que estos programas requieren, lo que genera una desconexión entre la formación académica y las exigencias del entorno laboral real.

Por otro lado, los métodos tradicionales utilizados en algunos procesos formativos no se adaptan al ritmo del aprendiz, lo cual impacta negativamente en la retención del conocimiento. En consecuencia, esta problemática afecta directamente la competitividad y empleabilidad del aprendiz egresado.

4.2 Solución propuesta

Solución propuesta: SENALingua		
Componente	**Función**	**Propósito**
Plataforma web/móvil	Acceso al sistema	Disponibilidad
IA adaptativa	Personalización	Mejor aprendizaje
Gamificación	Motivación	Retención
Evaluaciones	Medición	Seguimiento

SENALingua se plantea como una solución específica para el contexto formativo del SENA, integrando una plataforma digital que permite a los aprendices desarrollar competencias en inglés técnico aplicado al desarrollo de software.

En primer lugar, la accesibilidad web/móvil permite que el aprendiz interactúe con el sistema en cualquier momento, reforzando el aprendizaje autónomo. Además, la implementación de inteligencia artificial permite adaptar los contenidos según el nivel y progreso del usuario.

Asimismo, la gamificación responde a la necesidad de mantener la motivación en procesos de formación virtual, mientras que las evaluaciones permiten medir avances de forma objetiva. Por tanto, la solución está directamente alineada con las necesidades reales del entorno educativo y laboral.

5.0 Justificación del proyecto

5.1 Definición técnica

La justificación del proyecto SENALingua permite sustentar su implementación dentro del SENA como una solución pertinente a una necesidad formativa específica. En efecto, no se trata de incorporar tecnología por innovación, sino de resolver una limitación concreta en el aprendizaje del inglés técnico.

Además, esta sección permite evidenciar que el proyecto genera valor tanto para los aprendices como para la institución, alineándose con objetivos de calidad educativa y empleabilidad.

5.2 Justificación estructurada

Pedagógica		
Elemento	**Explicación**	**Impacto**
Aprendizaje contextual	Inglés enfocado en software	Mejor comprensión
Evaluación continua	No solo exámenes	Mejora progresiva

Justificación

SENALingua propone un aprendizaje contextualizado en situaciones reales del desarrollo de software, lo cual facilita que el aprendiz relacione el idioma con su campo de formación. Además, la evaluación continua permite evidenciar avances en competencias específicas, no solo en conocimiento general.

Por tanto, se fortalece el proceso formativo del aprendiz dentro del SENA.

Tecnológica		
Elemento	**Explicación**	**Impacto**
IA	Personalización	Eficiencia
Analítica	Seguimiento	Toma de decisiones

Justificación

El uso de inteligencia artificial en SENALingua permite adaptar contenidos según el desempeño del aprendiz, lo cual optimiza el proceso de aprendizaje. Además, la analítica del sistema permite a instructores identificar avances y dificultades.

En consecuencia, se mejora la eficiencia del proceso formativo mediante el uso de tecnología aplicada.

Estratégica		
Elemento	**Explicación**	**Impacto**
Empleabilidad	Mejora habilidades	Inserción laboral
Innovación	Uso de tecnología	Posicionamiento

Justificación

El proyecto SENALingua fortalece la formación del aprendiz SENA, alineándola con las exigencias del sector tecnológico, donde el inglés técnico es fundamental. Además, posiciona al SENA como una institución que integra soluciones innovadoras en sus procesos educativos.

Por tanto, genera valor institucional y mejora la competitividad de sus egresados.

Financiera		
Factor	**Explicación**	**Impacto**
Reducción de costos	Menos formación tradicional	Ahorro
Escalabilidad	Puede crecer	Mayor retorno
Digitalización	Automatización	Eficiencia

Justificación

SENALingua, al ser una plataforma digital, permite optimizar recursos institucionales al reducir la dependencia de métodos tradicionales de enseñanza. Además, su escalabilidad permite beneficiar a múltiples programas de formación sin incrementar significativamente los costos.

Por tanto, la inversión inicial se justifica en función del impacto que puede generar en un alto número de aprendices.

En consecuencia, el proyecto es financieramente sostenible dentro del contexto institucional.

Evaluación de utilidad

La utilidad de SENALingua radica en su capacidad de mejorar una competencia clave en los aprendices: el inglés técnico aplicado al software. En efecto, esto impacta directamente en su desempeño académico y en su futura inserción laboral.

Además, al tratarse de una solución reutilizable y escalable dentro del SENA, su valor aumenta con el tiempo.

Por tanto, el proyecto no solo es útil, sino estratégico para el fortalecimiento de la formación tecnológica institucional.

6.0 Alcance

6.1 Definición técnica

El alcance del proyecto define los límites funcionales, técnicos y operativos de SENALingua, estableciendo qué incluye y qué no incluye el sistema. Su propósito es garantizar una comprensión clara del producto a desarrollar, evitando desviaciones y asegurando el cumplimiento de los objetivos establecidos.

6.2 Alcance funcional

SENALingua contempla el desarrollo de una plataforma digital (web/móvil) orientada al aprendizaje de inglés técnico aplicado al desarrollo de software dentro del contexto formativo del SENA.

El sistema incluirá:

- Registro y autenticación de usuarios (aprendices e instructores)
- Diagnóstico inicial de nivel de inglés técnico
- Módulos de aprendizaje basados en terminología de software
- Sistema de ejercicios interactivos (lectura, escritura, comprensión)

- Evaluaciones continuas y seguimiento del progreso
- Implementación de elementos de gamificación (niveles, logros)
- Adaptación de contenidos mediante inteligencia artificial
- Panel de seguimiento para instructores

Justificación

El alcance funcional ha sido definido con base en las necesidades reales del aprendiz SENA en formación tecnológica. En efecto, cada funcionalidad responde a una limitación identificada, desde el diagnóstico inicial hasta el seguimiento del progreso.

Además, se priorizan funcionalidades que aportan valor directo al aprendizaje, evitando la sobrecarga de características innecesarias. Por tanto, se garantiza un sistema enfocado, usable y alineado con los objetivos del proyecto.

6.3 Alcance técnico

El sistema SENALingua será desarrollado como una aplicación web responsiva con posibilidad de acceso desde dispositivos móviles, utilizando tecnologías modernas de desarrollo frontend y backend.

El alcance técnico incluye:

- Arquitectura cliente-servidor
- Desarrollo de API para gestión de datos
- Base de datos para almacenamiento de usuarios y progreso
- Integración de modelos de inteligencia artificial para personalización
- Diseño de interfaz centrado en experiencia de usuario (UX/UI)
- Implementación de seguridad básica (autenticación y protección de datos)

Justificación

El alcance técnico se define bajo criterios de viabilidad y escalabilidad. En primer lugar, una arquitectura web permite acceso amplio sin requerir instalaciones complejas.

Además, el uso de inteligencia artificial responde a la necesidad de personalización del aprendizaje. Por otro lado, la implementación de buenas prácticas en desarrollo garantiza mantenibilidad y crecimiento del sistema.

En consecuencia, se establece una base tecnológica sólida y adaptable.

6.4 Alcance organizacional

El proyecto se desarrollará dentro del entorno académico del SENA, con la participación del equipo de desarrollo y la supervisión de instructores.

Incluye:

- Desarrollo como proyecto formativo
- Validación por parte de instructores
- Uso potencial dentro de programas de formación tecnológica

Justificación

Este alcance permite alinear el proyecto con los objetivos institucionales del SENA. Además, facilita su validación en un entorno real de uso.

Por tanto, se garantiza que el sistema no sea solo teórico, sino aplicable en el contexto educativo.

6.5 Exclusiones del alcance

El proyecto NO incluye:

- Certificación oficial de inglés
- Integración con plataformas externas (por ejemplo, LMS institucionales)
- Desarrollo de aplicación nativa (solo versión web responsiva)
- Soporte multilenguaje (enfocado únicamente en inglés técnico)

Justificación

Las exclusiones permiten delimitar el proyecto y evitar expansión innecesaria del alcance. En efecto, incluir estos elementos incrementaría la complejidad, el tiempo y los costos.

Por tanto, se prioriza el desarrollo de un producto funcional y viable dentro del contexto del proyecto formativo.

7.0 Requerimientos del sistema

7.1 Definición técnica

Los requerimientos del sistema describen de manera estructurada las funcionalidades y condiciones que debe cumplir SENALingua para satisfacer las necesidades del usuario. Se clasifican en requerimientos funcionales y no funcionales, siguiendo estándares de ingeniería de software.

7.2 Requerimientos funcionales

Gestión de usuarios	
Código	**Requerimientos**
RF-01	El sistema debe permitir el registro de usuarios (aprendices e instructores)
RF-02	El sistema debe permitir el inicio de sesión mediante credenciales
RF-03	El sistema debe gestionar roles de usuario (aprendiz/instructor)

Justificación

La gestión de usuarios es fundamental para personalizar la experiencia dentro de SENALingua. Además, permite diferenciar funciones entre aprendices e instructores.

Por tanto, garantiza control de acceso y organización del sistema.

Diagnóstico inicial	
Código	**Requerimiento**
RF-04	El sistema debe evaluar el nivel inicial de inglés técnico del usuario

RF-05	El sistema debe asignar contenidos según el resultado del diagnóstico

Justificación

El diagnóstico permite adaptar el aprendizaje desde el inicio. En efecto, evita que el usuario reciba contenido inadecuado a su nivel.

En consecuencia, mejora la eficiencia del proceso formativo.

Módulos de aprendizaje	
Código	**Requerimiento**
RF-06	El sistema debe ofrecer módulos de contenido enfocados en inglés técnico
RF-07	El sistema debe incluir ejercicios interactivos
RF-08	El sistema debe presentar contenido progresivo por niveles

Justificación

Estos módulos constituyen el núcleo del sistema. Además, el enfoque en contenido técnico asegura relevancia para el aprendiz SENA.

Por tanto, se garantiza un aprendizaje aplicable al contexto real.

Evaluación y seguimiento	
Código	**Requerimiento**
RF-09	El sistema debe evaluar el progreso del usuario
RF-10	El sistema debe almacenar resultados de evaluaciones
RF-11	El sistema debe mostrar el avance del usuario

Justificación

La evaluación permite medir resultados de aprendizaje. Asimismo, el seguimiento facilita la toma de decisiones tanto para el usuario como para el instructor.

En consecuencia, se fortalece el control del proceso formativo.

Gamificación	
Código	**Requerimiento**
RF-12	El sistema debe implementar niveles o logros
RF-13	El sistema debe otorgar retroalimentación motivacional

Justificación

La gamificación mejora la motivación del usuario. En efecto, incrementa la permanencia en la plataforma.

Por tanto, reduce la deserción en procesos virtuales.

Panel de instructor	
Código	**Requerimiento**
RF-14	El sistema debe permitir al instructor visualizar el progreso de los aprendices
RF-15	El sistema debe generar reportes básicos de desempeño

Justificación

Este componente permite integrar al instructor en el proceso. Además, facilita el acompañamiento del aprendizaje.

En consecuencia, mejora la calidad formativa.

7.3 Requerimientos no funcionales

Usabilidad	
Código	**Requerimiento**
RNF-01	El sistema debe ser intuitivo y fácil de usar
RNF-02	El sistema debe ser accesible desde diferentes dispositivos

Justificación

La usabilidad es clave para la adopción del sistema. En efecto, un sistema complejo reduce su uso.

Por tanto, se prioriza una experiencia clara y sencilla.

Rendimiento	
Código	**Requerimiento**
RNF-03	El sistema debe responder en tiempos adecuados (<3 segundos)

Justificación

El rendimiento impacta directamente en la experiencia del usuario. Además, tiempos de respuesta altos generan abandono.

En consecuencia, se garantiza eficiencia operativa.

Seguridad	
Código	**Requerimiento**
RNF-04	El sistema debe proteger la información de los usuarios
RNF-05	El sistema debe implementar autenticación segura

Justificación

La seguridad es fundamental en cualquier sistema que maneje datos. Además, garantiza confianza del usuario.

Por tanto, es un requisito obligatorio.

Escalabilidad	
Código	**Requerimiento**
RNF-06	El sistema debe permitir crecimiento en número de usuarios

Justificación

SENALingua está diseñado para ser utilizado por múltiples programas del SENA. En efecto, debe soportar crecimiento.

En consecuencia, se asegura sostenibilidad del sistema.

8.0 Arquitectura tecnológica

La arquitectura tecnológica define la estructura técnica mediante la cual se construirá el sistema SENALingua, estableciendo los lenguajes de programación, herramientas y componentes que permitirán el funcionamiento del software. La selección de tecnologías se basa en criterios de accesibilidad, facilidad de aprendizaje, compatibilidad entre herramientas y disponibilidad de documentación.

Se adopta una arquitectura **cliente – servidor,** que permite separar la interfaz de usuario de la lógica del sistema y del almacenamiento de datos, facilitando mantenimiento, escalabilidad y organización del código.

8.1 Frontend

a. Tecnologías

- Flutter
- Dart
- HTML (estructura web si aplica)
- CSS (estilos visuales)
- Material UI (componentes visuales)
- Figma (diseño de interfaces)

b. Funciones principales

- Construcción de interfaces gráficas del sistema
- Formularios de interacción con el usuario
- Visualización de resultados
- Navegación entre módulos
- Validación básica de datos ingresados
- Consumo de servicios del backend mediante API
- Adaptación visual para dispositivos móviles

Justificación

El frontend se construye utilizando Flutter debido a su facilidad de aprendizaje y capacidad de generar interfaces modernas de forma eficiente. Permite desarrollar aplicaciones con una sola base de código, reduciendo complejidad técnica para el equipo de aprendices. La estructura visual se diseña previamente en Figma para garantizar organización en la experiencia de usuario. La separación entre diseño y lógica facilita modificaciones futuras sin afectar el funcionamiento del sistema.

8.2 Backend

a. Tecnologías

- Python
- Framework Flask
- API REST
- JSON

b. Funciones principales

- Procesamiento de solicitudes del usuario
- Gestión de lógica del sistema
- Validación de datos
- Comunicación con la base de datos
- Integración con el módulo de IA
- Generación de respuestas del sistema
- Control de acceso a la información
- Gestión de servicios del sistema

Justificación

Python permite desarrollar la lógica del sistema de manera clara y organizada, facilitando la comprensión del código por parte de aprendices. Flask permite construir servicios web de forma sencilla sin requerir configuraciones complejas. La implementación de API REST permite conectar el frontend con el backend de forma estructurada, garantizando orden en el flujo de información del sistema.

8.3 Base de datos

a. Tecnologías

- MySQL
- SQL

b. Funciones principales

- Almacenamiento de información del sistema
- Registro de usuarios
- Gestión de datos lingüísticos
- Consulta de información estructurada

- Relación entre tablas del sistema
- Persistencia de información

Justificación

MySQL permite gestionar datos de forma estructurada y es ampliamente utilizado en entornos académicos. Su integración con Python es sencilla y cuenta con abundante documentación. La base de datos permite organizar la información de forma lógica, facilitando consultas eficientes y garantizando integridad de los datos del sistema.

8.4 Módulo de inteligencia artificial (Chatbot)

a. Tecnologías

- Python
- Librerías NLP básicas
- API de procesamiento de lenguaje natural

b. Funciones principales

- Interpretación de texto ingresado por el usuario
- Generación de respuestas automatizadas
- Procesamiento básico del lenguaje
- Apoyo en la traducción o interpretación de contenido
- Mejora de interacción del usuario con el sistema

Justificación

El uso de herramientas básicas de procesamiento de lenguaje permite implementar funcionalidades de interpretación de texto sin requerir configuraciones complejas. El enfoque se orienta

a una solución funcional alcanzable para aprendices, evitando dependencias técnicas avanzadas que puedan dificultar el desarrollo del proyecto.

8.5 Control de versiones

1. **Tecnologías**

- Git
- GitHub

2. **Funciones principales**

- Almacenamiento del código fuente
- Control de cambios en el proyecto
- Trabajo colaborativo
- Recuperación de versiones anteriores
- Organización del desarrollo

Justificación

El control de versiones permite mantener organización en el desarrollo del proyecto, facilitando el trabajo en equipo y evitando pérdida de información. GitHub permite almacenar el código en línea y mantener respaldo seguro del proyecto.

8.6 Despliegue del sistema

a. **Tecnologías**

- Render o Railway
- Hosting básico
- Dominio web

b. Funciones principales

- Publicación del sistema en internet
- Acceso institucional al sistema
- Pruebas en entorno real
- Disponibilidad del sistema para evaluación

Justificación

El uso de servicios de despliegue accesibles permite publicar el sistema sin requerir infraestructura compleja. Esto facilita la validación del funcionamiento del software en un entorno real y permite demostrar la operatividad del sistema desarrollado.

8.7 Estructura general de la arquitectura

El sistema se compone de tres capas principales:

- **Capa de presentación**

Interfaz visual con la que interactúa el usuario.

- **Capa lógica**

Procesamiento de información y reglas del sistema.

- **Capa de datos**

Almacenamiento estructurado de la información.

Esta organización permite mantener orden en el desarrollo, facilitar mantenimiento y mejorar la comprensión del funcionamiento del sistema.

9.0 Metodología de desarrollo

9.1 Definición técnica

La metodología de desarrollo establece el marco estructurado mediante el cual se planifica, ejecuta, controla y valida la construcción del sistema SENALingua, integrando prácticas ágiles que permiten gestionar la incertidumbre, optimizar recursos y asegurar la entrega incremental de valor.

9.2 Metodología seleccionada

Se adopta Scrum como marco de trabajo ágil, debido a su capacidad para gestionar proyectos con requerimientos evolutivos, como es el caso de SENALingua, donde intervienen variables pedagógicas, tecnológicas y de experiencia de usuario.

Justificación

Scrum permite trabajar bajo ciclos iterativos cortos, lo que facilita validar continuamente el sistema con usuarios reales (aprendices e instructores). Además, reduce el riesgo de desviación del producto, ya que cada incremento es evaluado antes de avanzar.

9.3 Estructuración operativa de SCRUM en SENALingua

- ✓ **Gestión del producto backlog**

El Product Backlog será definido y priorizado por el Product Owner, incluyendo requerimientos funcionales y no funcionales previamente establecidos (RF y RNF).

Características:

- Priorización basada en valor pedagógico y funcional
- Refinamiento continuo (Backlog Refinement)
- Descomposición en historias de usuario

Justificación

La gestión del backlog permite enfocar el desarrollo en funcionalidades críticas, como el diagnóstico inicial o la personalización del aprendizaje. Además, su actualización constante asegura que el sistema evolucione según las necesidades reales del SENA.

- ✓ **Planificación de sprints (Sprint planning)**

Cada sprint tendrá una duración estimada de 2 semanas, en las cuales se seleccionarán tareas priorizadas del backlog.

Actividades clave:

- Definición del objetivo del sprint (Sprint Goal)
- Selección de historias de usuario
- Estimación de esfuerzo (story points)
- Asignación de tareas al equipo

Justificación

La planificación permite establecer un enfoque claro por iteración. Además, la estimación de esfuerzo contribuye a una distribución equilibrada del trabajo.

En consecuencia, se optimiza el uso del tiempo y se reduce la incertidumbre operativa.

- ✓ **Desarrollo iterativo e incremental**

El desarrollo de SENALingua se realizará mediante incrementos funcionales, donde cada sprint entregará una parte operativa del sistema.

Ejemplo aplicado:

- Sprint 1: Autenticación de usuarios
- Sprint 2: Diagnóstico inicial
- Sprint 3: Módulos de aprendizaje
- Sprint 4: Evaluaciones y seguimiento

Justificación

Este enfoque permite validar progresivamente el sistema. Además, facilita detectar errores tempranos y realizar ajustes sin afectar el proyecto completo.

Por tanto, mejora la calidad y reduce riesgos.

- ✓ **Daily Scrum (Seguimiento operativo)**

Se realizarán reuniones diarias de máximo 15 minutos para seguimiento del equipo.

Enfoque:

- ¿Qué se hizo?
- ¿Qué se hará?
- Bloqueos identificados

Justificación

El seguimiento diario permite detectar problemas de forma temprana. Además, mejora la comunicación del equipo.

En consecuencia, se evita acumulación de errores y retrasos.

- ✓ **Sprint Review (Validación del incremento)**

Al finalizar cada sprint, se presentará el incremento funcional a los stakeholders.

Incluye:

- Demostración del sistema
- Validación de funcionalidades
- Recolección de retroalimentación

Justificación

La revisión permite validar que el sistema cumple con los requerimientos definidos. Además, integra la participación de instructores, asegurando pertinencia pedagógica.

Por tanto, se garantiza alineación con el objetivo del proyecto.

- ✓ **Sprint Retrospective (Mejora continua)**

Luego, cada sprint, el equipo evaluará su desempeño.

Enfoque:

- ¿Qué funcionó?
- ¿Qué no funcionó?
- ¿Qué mejorar?

Justificación

La retrospectiva permite optimizar el proceso de desarrollo. Además, fomenta mejora continua en el equipo.

En consecuencia, incrementa la eficiencia en iteraciones futuras.

9.4 Gestión de calidad dentro de Scrum

La calidad en SENALingua será integrada dentro de cada sprint, no como una fase final.

Prácticas aplicadas

- Validación continua (QA en cada iteración)
- Pruebas funcionales por módulo
- Revisión de cumplimiento de requerimientos

Justificación

Incorporar calidad de forma continua reduce el costo de corrección de errores. Además, asegura que cada incremento sea funcional.

Por tanto, se evita acumulación de fallos al final del proyecto.

9.5 Gestión de riesgos

Dentro de la metodología Scrum, los riesgos se gestionan de forma iterativa.

Riesgos identificados en SENALingua:

- Mala interpretación de requerimientos pedagógicos
- Fallas en la personalización (IA)
- Baja adopción por parte del usuario

Estrategias

- Validación constante con instructores
- Pruebas progresivas del sistema
- Ajustes basados en retroalimentación

Justificación

La gestión continua de riesgos permite reducir incertidumbre. Además, evita impactos críticos en etapas avanzadas.

En consecuencia, se mejora la estabilidad del proyecto.

9.6 Justificación integral de la metodología

La adopción de Scrum en SENALingua responde a la necesidad de desarrollar un sistema que combina componentes técnicos y pedagógicos, los cuales requieren validación constante.

En efecto, su enfoque iterativo permite adaptar el sistema a medida que se identifican nuevas necesidades. Además, la entrega incremental garantiza resultados visibles en corto plazo.

Por tanto, Scrum no solo optimiza el desarrollo, sino que asegura que el producto final sea funcional, usable y alineado con el contexto del SENA.

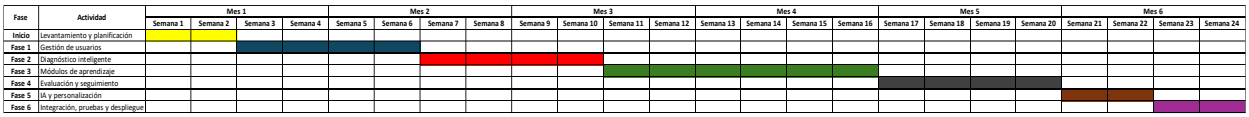
10. Cronograma del proyecto

10.1 Definición del horizonte temporal

El horizonte temporal de 6 meses (24 semanas) responde a una planificación estratégica que equilibra capacidad operativa del equipo, complejidad funcional del sistema y necesidad de validación progresiva. Desde la perspectiva de desarrollo de software, este periodo permite estructurar el trabajo en 12 sprints de 2 semanas, lo cual garantiza ciclos de entrega suficientemente cortos para inspección y ajuste, pero lo bastante amplios para generar incrementos funcionales estables. A nivel de gestión de proyectos, este horizonte asegura control sobre alcance, tiempo y calidad, mientras que, desde el enfoque financiero, permite distribuir el costo del desarrollo de forma eficiente, evitando sobrecostos derivados de retrabajos o mala definición inicial.

10.2 Estructura general del cronograma

Fase	Actividad	Requerimientos asociados	Duración	Responsable	Entregable
Inicio	Levantamiento y planificación	RF01–RF02, RNF01	2 semanas	PO / Scrum Master	Backlog validado
Fase 1	Gestión de usuarios	RF01–RF02, RNF02	4 semanas	Frontend / Backend	Sistema de autenticación
Fase 2	Diagnóstico inteligente	RF03–RF04	4 semanas	Backend	Evaluación diagnóstica
Fase 3	Módulos de aprendizaje	RF05–RF06, RNF03	6 semanas	Frontend	Contenido interactivo
Fase 4	Evaluación y seguimiento	RF07–RF08, RNF04	4 semanas	Backend / QA	Sistema evaluativo
Fase 5	IA y personalización	RF10, RNF06	2 semanas	Backend	Adaptación inteligente
Fase 6	Integración, pruebas y despliegue	RF09, RNF05	2 semanas	QA / Dev	Sistema final



Justificación del tiempo total del cronograma

La duración de 6 meses no responde únicamente a una estimación técnica, sino a una decisión fundamentada en la naturaleza del proyecto SENALingua. En primer lugar, el sistema no es un desarrollo convencional, ya que integra componentes pedagógicos, interacción de usuario y personalización mediante inteligencia artificial, lo cual incrementa significativamente la complejidad. En segundo lugar, el desarrollo requiere validación continua con enfoque educativo, lo que implica iteraciones adicionales para asegurar que los contenidos y funcionalidades realmente aporten valor al aprendiz.

Adicionalmente, desde el punto de vista financiero, extender el desarrollo a este periodo permite optimizar la inversión, ya que reduce el riesgo de fallos críticos y evita costos posteriores de corrección. En consecuencia, el cronograma no solo cubre construcción, sino también pruebas, ajustes y estabilización, garantizando un producto final robusto y sostenible.

10.3 Distribución por sprints

Fase 1 – Gestión de usuarios			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 1	RF01	Registro de usuarios	Formulario funcional
Sprint 2	RF02, RNF02	Autenticación y seguridad	Login seguro

Justificación

Se consolida la base del sistema, garantizando control de acceso y seguridad desde etapas tempranas.

Fase 2 – Diagnóstico del usuario			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 3	RF03	Evaluación diagnóstica	Prueba inicial
Sprint 4	RF04	Clasificación del usuario	Nivel asignado

Justificación

Permite establecer la base para la personalización del aprendizaje.

Fase 3 – Módulos de aprendizaje			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 5	RF03	Desarrollo de contenido	Lecciones
Sprint 6	RF04	Actividades interactivas	Ejercicios
Sprint 7	RNF03	Mejora UI/UX	Interfaz optimizada

Justificación

Se asigna mayor tiempo debido a la complejidad pedagógica y técnica del contenido.

Fase 4 – Evaluación y seguimiento			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 8	RF07	Evaluaciones	Pruebas funcionales
Sprint 9	RF08, RNF04	Seguimiento	Dashboard

Justificación

Se garantiza medición del aprendizaje y control del progreso del usuario.

Fase 5 – Inteligencia artificial			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 10	RF10, RNF06	Personalización	Motor adaptativo

Justificación

Esta fase eleva el valor del sistema, permitiendo aprendizaje adaptativo.

Fase 6 – Integración y calidad			
Sprint	**Actividad**	**Enfoque**	**Entregable**
Sprint 11	RF09	Personalización	Sistema unificado
Sprint 12	RNF05	Pruebas y despliegue	Sistema final

Justificación

Se asegura estabilidad, calidad y funcionamiento integral antes de la entrega.

10.3.1 Explicación de la distribución de los sprints

La distribución de los sprints está diseñada bajo un criterio de dependencia funcional y generación progresiva de valor. Inicialmente, el Sprint 0 se orienta a la estructuración del backlog, lo

cual es fundamental para asegurar que el desarrollo posterior esté alineado con los requerimientos reales; sin esta base, el proyecto perdería control de alcance. Posteriormente, los primeros sprints se enfocan en la gestión de usuarios, ya que la autenticación es un prerrequisito técnico para cualquier interacción dentro del sistema, permitiendo identificar al usuario y habilitar la personalización.

Una vez establecida esta base, los siguientes sprints se concentran en el diagnóstico del usuario, lo cual responde directamente al enfoque adaptativo de SENALingua; sin esta etapa, no sería posible ofrecer un aprendizaje diferenciado. A continuación, se asigna un bloque más amplio de sprints al desarrollo de los módulos de aprendizaje, debido a que este componente constituye el núcleo del sistema, no solo por su complejidad técnica, sino por la necesidad de estructurar contenidos pedagógicos coherentes y funcionales.

Posteriormente, los sprints dedicados a evaluación y seguimiento permiten cerrar el ciclo de aprendizaje, incorporando medición del desempeño y visualización del progreso, lo cual es esencial tanto para el usuario como para el control del sistema. En las etapas finales, se introduce la implementación de inteligencia artificial, la cual se ubica estratégicamente después de contar con datos y funcionalidades base, garantizando así una personalización efectiva y no teórica. Finalmente, los últimos sprints se destinan a integración, pruebas y despliegue, asegurando que todos los componentes funcionen de manera conjunta, cumpliendo criterios de calidad, estabilidad y rendimiento.

Desde una perspectiva de gestión, esta secuencia reduce riesgos al priorizar primero los componentes críticos; desde el desarrollo, asegura dependencias técnicas correctas; y desde el análisis financiero, optimiza la inversión al evitar reprocesos y garantizar que cada fase entregue valor tangible.

11. Propuesta financiera

11.1 Mano de obra del proyecto (Costo directo principal)

COSTOS DE PERSONAL			
ROL	**HORAS**	**TARIFA HORA (COP)**	**SUBTOTAL**
Analista de requisitos	50	18.000,00	900.000
Arquitecto de software	25	20.000,00	500.000
Desarrollador Backend	120	18.000,00	2.160.000
Desarrollador Frontend	120	18.000,00	2.160.000
Ingeniero de base de datos	40	18.000,00	720.000
Ingeniero IA / Chatbot	60	20.000,00	1.200.000
Diseñador UX/UI	60	18.000,00	1.080.000
QA / Tester	80	17.000,00	1.360.000
Scrum Master	40	20.000,00	800.000
Product Owner	30	20.000,00	600.000
TOTAL MANO DE OBRA			**11.480.000**

Justificación

El costo de mano de obra representa el componente principal del presupuesto debido a que el desarrollo de software constituye una actividad intensiva en conocimiento especializado. El proyecto requiere la participación coordinada de diferentes perfiles con funciones claramente definidas dentro del ciclo de vida de desarrollo, lo cual garantiza trazabilidad de los procesos, cumplimiento de requisitos funcionales y calidad del producto final.

La asignación de horas responde a la complejidad funcional del sistema SENALingua, el cual contempla componentes de interfaz de usuario, procesamiento de información, almacenamiento estructurado de datos, integración de servicios y validación funcional del sistema mediante pruebas técnicas.

La distribución del tiempo de trabajo permite cubrir las fases de análisis, diseño, desarrollo, pruebas, documentación y coordinación del proyecto. Esta organización asegura que cada componente

del sistema sea construido de manera controlada y verificable, evitando reprocesos que puedan generar sobrecostos o retrasos en la entrega final.

El número de horas asignadas refleja una estimación razonable basada en el alcance funcional del sistema, permitiendo cumplir con estándares de calidad académica y garantizando que el producto final pueda ser utilizado por la institución sin requerir desarrollos adicionales inmediatos.

La participación de roles de coordinación y definición funcional permite asegurar coherencia en los entregables, evitando inconsistencias técnicas o desviaciones en el alcance del proyecto. Esto contribuye a optimizar el uso del presupuesto al reducir riesgos de reprocesos o ajustes posteriores.

11.2 Costos del hardware (CAPEX)

COSTOS DE HARDWARE (CAPEX)			
RECURSO	**CANTIDAD**	**VALOR UNITARIO**	**TOTAL**
Laptop desarrollo gama media	2	2.200.000	4.400.000
Smartphone pruebas	1	700.000	700.000
Router básico	1	150.000	150.000
Disco externo respaldo	1	300.000	300.000
TOTAL HARDWARE			**5.550.000**

Justificación

El componente de hardware corresponde a la infraestructura mínima necesaria para garantizar el desarrollo, pruebas y validación del sistema en condiciones reales de funcionamiento. La selección de equipos responde a criterios de funcionalidad, compatibilidad tecnológica y vida útil adecuada para proyectos académicos.

Los equipos de cómputo permiten ejecutar herramientas de programación, simuladores, entornos de prueba y sistemas de control de versiones, los cuales son indispensables para la

construcción del software. La disponibilidad de dispositivos físicos garantiza que el sistema pueda ser validado en escenarios reales de uso, lo cual mejora la calidad del producto final.

El dispositivo móvil contemplado dentro del presupuesto permite verificar el comportamiento del sistema en entornos de interacción directa con el usuario final, lo cual resulta fundamental para asegurar accesibilidad y funcionamiento adecuado de la aplicación.

Los dispositivos de conectividad y almacenamiento permiten asegurar la disponibilidad de los recursos digitales del proyecto, facilitando la realización de copias de seguridad y evitando pérdida de información durante el proceso de desarrollo.

La inversión en hardware se mantiene dentro de un rango razonable, evitando la adquisición de equipos con especificaciones superiores a las requeridas por el alcance funcional del sistema.

11.3 Costos de software y servicios (OPEX)

COSTOS DE SOFTWARE Y SERVICIOS (OPEX)		
SOFTWARE	**TIPO**	**VALOR**
Ubuntu / Linux	Open Source	0
Python	Open Source	0
Flutter SDK	Open Source	0
MySQL	Open Source	0
Visual Studio Code	Free	0
GitHub	Free	0
Figma	Free	0
Postman	Free	0
Firebase Spark	Free	0
Hosting básico	Free trial	0
Dominio web anual	Pago	80.000
TOTAL SOFTWARE		**80.000**

Justificación

El proyecto prioriza el uso de herramientas tecnológicas de acceso libre o bajo costo, lo cual permite optimizar el uso del presupuesto institucional sin afectar la calidad técnica del sistema desarrollado.

El uso de software de código abierto garantiza independencia tecnológica, posibilidad de mantenimiento futuro y reducción de costos asociados a licenciamiento. Esta decisión permite que la institución pueda continuar utilizando el sistema sin incurrir en gastos recurrentes elevados.

El costo asociado al dominio web corresponde a un gasto necesario para facilitar el acceso al sistema y permitir su identificación dentro de un entorno digital institucional. Este elemento contribuye a la disponibilidad del sistema y facilita su implementación en un entorno controlado.

La selección de herramientas tecnológicas responde a criterios de estabilidad, soporte comunitario y compatibilidad con proyectos académicos de desarrollo de software.

11.4 Gastos operativos del proyecto

GASTOS OPERATIVOS DEL PROYECTO	
CONCEPTO	**VALOR**
Energía proporcional proyecto	450.000
Internet proporcional	420.000
Papelería y documentación	120.000
Administración y coordinación	300.000
TOTAL GASTOS OPERATIVOS	**1.290.000**

Justificación

Los gastos operativos contemplan los recursos necesarios para garantizar la ejecución continua del proyecto durante el periodo de desarrollo. Estos costos corresponden al consumo proporcional de servicios básicos utilizados en la ejecución de actividades técnicas y administrativas.

El consumo de energía y conectividad permite el funcionamiento de los equipos tecnológicos requeridos para el desarrollo del software, así como el acceso a repositorios digitales y servicios necesarios para la construcción del sistema.

Los recursos destinados a gestión administrativa permiten asegurar coordinación del proyecto, organización de la documentación y control de avances, lo cual facilita el cumplimiento de los objetivos establecidos.

La inclusión de costos asociados a documentación garantiza la correcta presentación de los entregables requeridos por la institución, asegurando claridad en los procesos realizados durante el desarrollo del proyecto.

11.5 Costos indirectos obligatorios

COSTOS INDIRECTOS OBLIGATORIOS		
Concepto	Base cálculo	Valor
Contingencia técnica 5%	Costos directos	915.000
Gestión documental	Fijo	250.000
Control de calidad	Fijo	300.000
TOTAL COSTOS INDIRECTOS		**1.465.000**

Justificación

Los costos indirectos permiten fortalecer la estabilidad del proyecto mediante la previsión de posibles variaciones en los costos o necesidades adicionales que puedan surgir durante el desarrollo del sistema.

La inclusión de una reserva para contingencias permite cubrir ajustes menores sin afectar la continuidad del proyecto, garantizando que las actividades puedan completarse dentro del plazo establecido.

La gestión documental permite asegurar que los productos generados durante el desarrollo del sistema cuenten con trazabilidad, organización y coherencia estructural, facilitando su revisión por parte de la institución.

El control de calidad permite verificar que el sistema cumpla con los requisitos establecidos, asegurando consistencia funcional y estabilidad técnica.

Estos costos contribuyen a fortalecer la confiabilidad del proyecto y a reducir riesgos asociados a fallas técnicas o inconsistencias en la documentación.

11.6 Utilidad académica del proyecto

UTILIDAD ACADÉMICA DEL PROYECTO	
Concepto	Valor
Utilidad académica 8%	1.582.000

Justificación

El margen considerado dentro del presupuesto permite mantener sostenibilidad en la ejecución del proyecto y cubrir esfuerzos adicionales que puedan surgir durante el proceso de desarrollo.

Este componente no corresponde a un beneficio comercial, sino a un mecanismo que permite absorber variaciones en tiempos de trabajo, ajustes técnicos o actividades adicionales que no hayan sido previstas inicialmente.

La inclusión de este margen contribuye a mantener equilibrio financiero en el proyecto y facilita la finalización completa de los entregables establecidos.

12. Forma de pago

FORMA DE PAGO (ESTRUCTURA FINANCIERA POR FASES)			
FASE	**ENTREGABLE DESEADO**	**%**	**VALOR**
Inicio del proyecto	Acta de inicio firmada	15%	3.217.050
Análisis y requisitos	Documento de requisitos aprobado	20%	4.289.400
Diseño del sistema	Arquitectura y prototipos UI	15%	3.217.050
Desarrollo backend	API funcional	15%	3.217.050
Desarrollo frontend	Interfaces funcionales	15%	3.217.050
Integración IA	Módulo chatbot operativo	10%	2.144.700
Pruebas y validación	Informe de pruebas	5%	1.072.350
Entrega final	Documentación completa	5%	1.072.350
TOTAL FORMA DE PAGO		**100%**	**21.447.000**

Justificación

La distribución porcentual de pagos permite mantener equilibrio entre riesgo técnico y avance funcional del sistema, asegurando que cada etapa del proyecto genere resultados verificables antes de continuar con la siguiente fase.

El anticipo asociado al inicio del proyecto permite disponer de los recursos necesarios para la organización inicial, definición de lineamientos técnicos y planificación detallada del desarrollo. Este componente facilita la estructuración adecuada del proyecto y permite establecer bases sólidas para la ejecución.

El porcentaje asignado a la fase de análisis y requisitos responde a la importancia estratégica de definir correctamente el alcance funcional del sistema, reduciendo la probabilidad de ajustes posteriores que puedan afectar el cronograma o la estabilidad del desarrollo.

Las fases de diseño, desarrollo backend y desarrollo frontend presentan una distribución equilibrada debido a que constituyen el núcleo técnico del sistema. Estas etapas concentran la mayor carga de trabajo especializada, permitiendo construir la estructura funcional del software.

La fase de integración del componente de inteligencia artificial requiere validaciones adicionales relacionadas con el procesamiento de información y comportamiento del sistema, lo cual justifica la asignación de recursos específicos para asegurar su correcto funcionamiento.

El porcentaje correspondiente a pruebas y validación permite garantizar que el sistema cumpla con los requisitos establecidos, verificando estabilidad funcional, coherencia de resultados y consistencia en la experiencia de usuario.

El pago final asegura la entrega completa de la documentación técnica y funcional requerida por la institución, garantizando disponibilidad de información suficiente para uso, mantenimiento y evaluación académica del sistema desarrollado.

La estructura definida permite mantener trazabilidad del proyecto, facilitar seguimiento institucional y asegurar cumplimiento de los objetivos establecidos dentro del alcance aprobado.

13. Entregables

La definición de entregables establece los productos verificables que evidencian el cumplimiento del alcance funcional del sistema SENALingua, garantizando que la institución disponga de los elementos necesarios para comprender, utilizar, evaluar y mantener la solución tecnológica desarrollada. Cada entregable responde a criterios de trazabilidad, claridad técnica y coherencia estructural, permitiendo validar que el sistema cumple los objetivos planteados en el levantamiento de requerimientos.

13.1 Documento de requisitos

Descripción

Documento estructurado que consolida las necesidades funcionales y no funcionales del sistema, definiendo el alcance, comportamiento esperado, restricciones técnicas y criterios de aceptación del software.

Contenido mínimo

- Introducción
- Objetivo del sistema
- Alcance del proyecto
- Definición de usuarios del sistema
- Requisitos funcionales
- Requisitos no funcionales
- Casos de uso
- Reglas de negocio
- Criterios de aceptación
- Glosario de términos

Justificación

Este documento permite establecer una base clara y verificable sobre lo que el sistema debe realizar, evitando interpretaciones ambiguas durante el desarrollo. La consolidación formal de los requisitos garantiza coherencia entre las necesidades identificadas y las funcionalidades implementadas, reduciendo la probabilidad de inconsistencias técnicas o desviaciones en el alcance del proyecto. La definición estructurada de los requisitos facilita la validación institucional del sistema, permitiendo

comprobar que la solución desarrollada responde a los objetivos planteados. Asimismo, el documento permite mantener trazabilidad entre las funcionalidades implementadas y las necesidades identificadas inicialmente, contribuyendo a la organización del proceso de desarrollo y a la claridad en la evaluación del proyecto.

13.2 Código fuente

Descripción

Conjunto de archivos digitales que contienen la lógica de funcionamiento del sistema, organizados de manera estructurada y documentada, permitiendo su ejecución, revisión y mantenimiento futuro.

Contenido mínimo

- Código backend
- Código frontend
- Configuración del entorno
- Scripts de base de datos
- Estructura de carpetas organizada
- Control de versiones
- Comentarios técnicos relevantes

Justificación

El código fuente constituye el componente principal del sistema desarrollado, permitiendo que la institución disponga del producto tecnológico completo y funcional. La organización estructurada del código facilita su comprensión, modificación y actualización futura, permitiendo que el sistema pueda mantenerse vigente sin depender de los desarrolladores iniciales. La inclusión de comentarios técnicos

contribuye a la interpretación del funcionamiento interno del sistema, facilitando la continuidad del proyecto en escenarios de mejora o ampliación de funcionalidades. La disponibilidad del código garantiza transparencia en el proceso de desarrollo y permite verificar que la solución tecnológica corresponde a los requerimientos establecidos.

13.3 Manual técnico

Descripción

Documento que describe la estructura interna del sistema, tecnologías utilizadas y procedimientos necesarios para su instalación, configuración y despliegue en un entorno tecnológico.

Contenido mínimo

- Arquitectura del sistema
- Tecnologías utilizadas
- Requisitos del entorno
- Instalación del sistema
- Configuración inicial
- Estructura de componentes
- Integraciones del sistema
- Procedimiento de despliegue
- Recomendaciones de mantenimiento

Justificación

El manual técnico permite que el sistema pueda ser comprendido desde una perspectiva estructural, facilitando su instalación en diferentes entornos tecnológicos y asegurando su continuidad operativa. La documentación técnica contribuye a que el sistema pueda mantenerse funcional a largo

plazo, permitiendo realizar ajustes o mejoras sin afectar su estabilidad. La descripción detallada de la arquitectura facilita la identificación de los componentes del sistema y su relación funcional, lo cual mejora la capacidad de administración y control del software por parte de la institución. Este documento garantiza que el conocimiento técnico generado durante el desarrollo no se pierda y pueda ser reutilizado en procesos futuros.

13.4 Manual de usuario

Descripción

Documento orientado a los usuarios finales del sistema, que explica de manera clara el funcionamiento de la aplicación y los procedimientos necesarios para utilizar sus funcionalidades principales.

Contenido mínimo

- Introducción al sistema
- Acceso a la aplicación
- Descripción de funcionalidades
- Procedimientos de uso
- Recomendaciones de uso
- Solución de situaciones comunes

Justificación

El manual de usuario facilita la adopción del sistema por parte de los usuarios finales, permitiendo comprender su funcionamiento sin requerir conocimientos técnicos especializados. La explicación clara de las funcionalidades contribuye a mejorar la experiencia de uso y reduce la probabilidad de errores durante la interacción con el sistema. La disponibilidad de este documento

permite que el sistema pueda ser utilizado de manera autónoma, facilitando su implementación dentro del entorno institucional. La orientación estructurada del manual contribuye a optimizar el uso de la herramienta y a fortalecer la accesibilidad del sistema desarrollado.

13.5 Informe de pruebas

Descripción

Documento que presenta la verificación del funcionamiento del sistema mediante la ejecución de pruebas que permiten comprobar el cumplimiento de los requisitos establecidos.

Contenido mínimo

- Plan de pruebas
- Estrategia de validación
- Pruebas funcionales
- Pruebas de integración
- Pruebas de usabilidad
- Registro de resultados
- Evidencias de funcionamiento
- Validación de cumplimiento de requisitos

Justificación

El informe de pruebas permite demostrar que el sistema cumple con los criterios de funcionamiento establecidos durante la definición de requisitos. La ejecución de pruebas permite identificar posibles inconsistencias y asegurar que el sistema responda adecuadamente en condiciones normales de uso. La documentación de resultados proporciona evidencia verificable del comportamiento del sistema, facilitando su evaluación y aprobación institucional. La validación

funcional contribuye a garantizar estabilidad operativa y coherencia en la interacción del usuario con la aplicación, fortaleciendo la confiabilidad del producto final.

Conclusión

La estructura de la propuesta técnica del proyecto SENALingua permite evidenciar una planificación organizada de los elementos necesarios para el desarrollo de una herramienta digital orientada al fortalecimiento del aprendizaje del idioma inglés. La definición clara de los requerimientos, la selección de tecnologías adecuadas, la organización de la arquitectura del sistema y la estructuración de los entregables permiten establecer una base coherente para la construcción del producto.

La articulación entre los componentes técnicos, funcionales y financieros permite mantener correspondencia entre el alcance del proyecto y los recursos disponibles, garantizando viabilidad en su desarrollo. La organización de los elementos presentados facilita la comprensión del sistema y permite verificar que la solución propuesta responde a las necesidades identificadas.

La propuesta permite establecer lineamientos claros para la construcción de un sistema funcional y documentado, asegurando que el resultado final pueda ser utilizado dentro del entorno institucional como apoyo al proceso de aprendizaje del idioma inglés. La estructura definida contribuye a mantener coherencia en el desarrollo del proyecto y facilita su evaluación, validación y posible continuidad.